

Case Study for

Day 1: Azure Compute and Networking Components

Problem Statement:

A rapidly growing e-commerce company, "**ShopEase**," is facing several challenges with its existing on-premises infrastructure. During peak sales periods, their website experiences significant traffic surges, leading to performance degradation and downtime. Additionally, they are struggling with scaling their compute resources to meet demand, resulting in lost revenue and a poor customer experience. The IT team also finds it challenging to manage high availability and ensure business continuity during unexpected hardware failures.

The company has decided to migrate its core e-commerce application to Azure. The goal is to:

1. Ensure high availability and scalability for their web and database tiers.
2. Distribute traffic efficiently across instances and ensure that there is no downtime during peak periods.
3. Optimize costs by dynamically scaling compute resources.
4. Securely connect the application to the internal database and minimize exposure to the public internet.

Objective:

Design a cloud-based solution using **Azure Compute** and **Networking Components** to ensure that ShopEase can scale its services dynamically and handle traffic peaks while ensuring high availability and disaster recovery.

Proposed Solution Using Azure Services:

To address ShopEase's challenges, you will design and implement a solution leveraging Azure Virtual Machines, Availability Sets, VM Scale Sets, Load Balancers, and Virtual Networks.

Solution Breakdown:

1. **Azure Virtual Machines for the Web Application:**
 - **Step 1:** Deploy multiple **Azure Virtual Machines (VMs)** running the company's web application.
 - Select the appropriate **VM size** based on the anticipated traffic and resource requirements.
 - Deploy a **Windows Server** or **Linux-based VM**, depending on the application.

- **Step 2:** Ensure full control over the virtual machines by configuring operating system settings, software, and patches.

2. High Availability with Availability Sets:

- **Step 3:** To ensure that at least one VM is always available during maintenance or hardware failure, place the VMs into an **Azure Availability Set**.
 - The VMs in the Availability Set will be distributed across multiple **update domains** and **fault domains**, reducing the impact of hardware or software failures.
 - This configuration provides a **99.95% SLA** from Microsoft for VM availability.
- **Step 4:** Enable **Azure Backup** for regular VM backups to provide disaster recovery in case of data loss or system failure.

3. Scalability with Virtual Machine Scale Sets:

- **Step 5:** Implement an **Azure Virtual Machine Scale Set** to automatically scale the number of VM instances based on demand (e.g., CPU usage or incoming requests).
 - Configure autoscaling rules, such as increasing the number of VMs during peak times (e.g., Black Friday sales).
 - Use a minimum of two instances to ensure redundancy, with a maximum cap based on estimated peak traffic.
- **Step 6:** Test the autoscaling rules by simulating high traffic loads and verifying that additional VMs are automatically provisioned and deprovisioned.

4. Traffic Distribution with Azure Load Balancer:

- **Step 7:** Deploy an **Azure Load Balancer** to distribute incoming traffic evenly across all VM instances in the scale set or Availability Set.
 - Use the **public load balancer** for external traffic coming from customers accessing the website.
 - Set up health probes to monitor the health of VMs, ensuring traffic is only routed to healthy instances.
- **Step 8:** For internal communication (e.g., between the web and database tiers), use an **internal load balancer** to handle traffic securely within the VNet without exposing endpoints to the internet.

5. Networking and Security with Virtual Networks (VNet) and NSGs:

- **Step 9:** Create an **Azure Virtual Network (VNet)** to host the VMs, allowing for secure and private communication between the web servers, database, and other resources.

- Subnet the VNet into two parts: one for the web tier (VMs running the website) and one for the database tier.
- **Step 10:** Implement **Network Security Groups (NSGs)** to control inbound and outbound traffic to and from the web tier and database tier.
 - Allow traffic on **port 443** (HTTPS) for external web traffic and deny all other public ports.
 - Enable **VNet Peering** to securely connect the VNet hosting the web application to the VNet hosting the database.

6. Disaster Recovery and Load Balancing with Azure Traffic Manager:

- **Step 11:** For geo-redundancy and disaster recovery, implement **Azure Traffic Manager**.
 - Set up **priority routing** to route traffic to the primary region and failover to the secondary region in case of an outage.
 - Configure Traffic Manager profiles to route traffic based on geographic location to provide the lowest latency for global customers.

Steps for Implementation:

1. Provision Virtual Machines:

- Navigate to the Azure portal, create the necessary VMs, and configure them with the required software stack for the e-commerce application.
- Ensure that the VMs are connected to the VNet created earlier.

2. Configure Availability Sets:

- Create an Availability Set and ensure that the VMs are spread across multiple update and fault domains.
- Associate the VMs with the Availability Set during the VM creation process.

3. Set Up the VM Scale Set:

- Deploy a VM Scale Set that can scale based on predefined CPU or network metrics.
- Configure autoscaling rules using the Azure portal, and set the minimum and maximum instance thresholds based on traffic predictions.

4. Deploy Load Balancer:

- Create and configure a Load Balancer to distribute incoming web traffic across the VMs.

- Define health probes to monitor VM availability.

5. **Configure Virtual Networks (VNet):**

- Create a VNet with separate subnets for the web tier and database tier.
- Apply NSGs to secure the network and control access to the web servers.

6. **Implement Traffic Manager:**

- Set up Azure Traffic Manager with failover routing to handle traffic in the event of regional outages.
- Define the primary and secondary endpoints for the e-commerce application.

Outcome:

By implementing this cloud-based architecture on Azure, ShopEase can ensure:

- **High availability** through the use of Availability Sets and Traffic Manager.
- **Scalability** via VM Scale Sets to handle traffic surges during sales events.
- **Cost-efficiency** by scaling resources dynamically based on traffic.
- **Enhanced security** by leveraging VNets, NSGs, and Load Balancers to control access and manage traffic securely.

Key Takeaways:

- Using **Azure Virtual Machines** with **Availability Sets** ensures minimal downtime.
- **VM Scale Sets** allow dynamic scaling to meet peak demand, ensuring optimal performance and cost management.
- **Azure Load Balancer** distributes traffic efficiently, while **Traffic Manager** adds a layer of global redundancy and load distribution.
- Secure communication is achieved through **Virtual Networks** and **NSGs**, minimizing exposure to the public internet.

This case study provides a real-world scenario where students can apply their understanding of Azure Compute and Networking Components to build a scalable, secure, and highly available infrastructure for a modern cloud-based e-commerce platform.